GoldenSetAuditor

Interview Defense Guide — v0.2.0

Evaluation dataset quality auditor for LLM / RAG applications

1. What Is GoldenSetAuditor?

GoldenSetAuditor audits golden evaluation datasets for LLM and RAG applications before benchmark scores are published. It does not score model outputs. It checks whether the dataset being scored against is trustworthy.

Seven checks grouped into three categories:

Group	Check	Can FAIL?	Algorithm
Structural Integrity	Conflicting labels	Yes — FAIL	Exact match on normalised input; answers differ
Structural Integrity	Exact duplicates	No — WARN	Exact match on normalised input; answers identical
Content Quality	Weak expected answer	No — WARN	Meaningful token count after stopword filter
Content Quality	Ambiguous prompt	No — WARN	Anaphoric ref regex + multi-question mark heuristic
Content Quality	Over-easy example	No — WARN	Token Jaccard(prompt, answer) >= threshold
Content Quality	Near-duplicate scan	No — WARN / INSUFFICIENT_INPUT	Pairwise token Jaccard O(n²), capped at max_pairwise_checks
Coverage	Category coverage	No — WARN / INSUFFICIENT_INPUT	Count per category vs min_category_count

Only conflicting labels produces FAIL because that is the one structural violation with no interpretation that resolves it: the ground truth is undefined.

2. Core Algorithmic Implementation: Token Jaccard

Token Jaccard is the shared primitive underpinning near-duplicate, over-easy, and indirectly the ambiguous prompt check. The implementation:

(1) Extract tokens: re.findall(r'[a-zA-Z0-9]+', text.lower())

(2) Filter stopwords: remove closed-class words (the, a, is, and, what, how, ...) and single-character tokens

(3) Jaccard = |A ∩ B| / |A ∪ B| over the token sets

Stopword removal is critical. Without it, two prompts that share only 'what is the' would score ~0.6 Jaccard on short prompts, generating false near-duplicate warnings. Filtering to content words makes the score reflect semantic proximity rather than syntactic pattern matching.

The same tokeniser is used for both near-duplicate (pairwise prompt comparison) and over-easy (prompt vs answer comparison) to ensure consistent behaviour across checks.

3. Hard Interview Questions

Q1: Token Jaccard is order-insensitive and ignores synonyms. When does this matter most?

A: It matters most in two cases. First, semantic near-duplicates: 'How many vacation days do employees receive?' and 'What is the employee vacation day allowance?' share few token overlap but are nearly identical in meaning. Jaccard would not flag these; sentence embeddings would. Second, paraphrases as intentional test cases: a team may deliberately include 'Can employees carry over PTO?' and 'Is PTO carryover permitted?' to test whether the model is robust to phrasing variation. Jaccard would flag these as near-duplicates incorrectly. Both cases are why the tool returns WARN, not FAIL, and explicitly asks the reviewer to confirm whether a pair is an accidental duplicate or an intentional paraphrase.

Q2: LLM-as-judge vs golden set evaluation — what does each approach catch that the other misses?

A: Golden set evaluation catches exact failures against a predefined reference: the model either matches the expected answer or doesn't. It is reproducible, cheap, and auditable. It misses cases where the model's answer is correct but differently phrased from the reference (false negatives on exact match), and where the reference answer is wrong or outdated (false negatives everywhere). LLM-as-judge evaluation can assess semantic correctness, fluency, and relevance without exact-match constraints. It catches cases where multiple valid phrasings exist. It misses consistent model failures that happen to sound plausible (LLM judges are also susceptible to confident-sounding wrong answers), and it introduces variability from the judge model's own biases and prompt sensitivity. GoldenSetAuditor addresses the golden set path only. The validity of the judge model is a separate concern.

Q3: Pretraining contamination — the model may have seen the golden set during training. How would you detect it?

A: Pretraining contamination means the model has memorised the golden set's question-answer pairs rather than learning to reason. It produces inflated benchmark scores. Detection approaches: (a) n-gram overlap between golden set questions and known public datasets — a long n-gram match suggests the question appeared in a pretraining corpus; (b) membership inference attacks — probe the model's token log-probabilities on golden set examples vs held-out examples; higher log-probability on golden set suggests memorisation; (c) canary strings — insert synthetic question-answer pairs into the golden set before any public release and check if the model can reproduce them. GoldenSetAuditor v0 does not implement contamination detection — it would require access to the model, not just the dataset. It is a documented roadmap item.

Q4: Your near-duplicate scan is $O(n^2)$. What happens at 10,000 examples?

A: 10,000 examples produce ~50 million pairs. At ~100,000 Jaccard comparisons per second in Python, that is approximately 8 minutes of wall time — impractical. The max_pairwise_checks cap (default 25,000 pairs, i.e., ~225 examples) returns INSUFFICIENT_INPUT rather than running

indefinitely or crashing. The correct solution for large golden sets is approximate nearest neighbour: encode prompts with a sentence encoder, index with FAISS or Annoy, and query each prompt against its k nearest neighbours. This brings complexity to O(n log n) with a constant factor from the encoder. This is the primary roadmap item for scale.

Q5: What if both expected answers in a conflict are valid? Is FAIL too harsh?

A: This is a real concern for multi-answer questions: 'What year was the company founded?' might legitimately have two answers if the company was refounded after bankruptcy. However, the right response is not to downgrade to WARN — it is to resolve the ambiguity in the golden set itself by either picking a canonical answer and annotating the rationale, or removing the example. A golden set with two valid contradictory expected answers for the same prompt cannot produce a meaningful evaluation: the model will be penalised for giving the 'wrong' valid answer. FAIL is the correct status because the data integrity issue must be resolved before the example is safe to use, regardless of whether both answers are defensible.

Q6: You check if the expected answer is too short. Do you check if it's correct?

A: No, and this is an explicit limitation documented in the truth boundary. FeatureLeakageLens checks structural and statistical properties of the DataFrame. Factual correctness of expected answers requires domain knowledge: a human expert must verify that 'Employees may carry over 5 PTO days' is currently true for this company. An auditor that claims to verify factual correctness would need to be a domain-expert system with access to the authoritative source documents — a different product entirely. The weak expected answer check catches structural incompleteness (too short to be useful), not factual inaccuracy.

Q7: What evaluation metric should you use with a golden set — exact match, ROUGE, or semantic similarity?

A: It depends on the task. Exact match is appropriate for factoid questions with a canonical short answer ('What is the capital of France?' → 'Paris'). It is too strict for open-ended questions where multiple valid phrasings exist. ROUGE-L measures longest common subsequence overlap and is appropriate for summarisation tasks where the expected answer is a multi-sentence summary. It is insensitive to word order and rewards verbosity. Semantic similarity (cosine similarity of sentence embeddings) is appropriate when meaning matters more than exact phrasing, but requires choosing an embedding model and is not deterministic across model versions. GoldenSetAuditor does not choose a metric — it audits the dataset quality that determines whether any of these metrics is trustworthy.

Q8: What about multilingual golden sets? Does token Jaccard still work?

A: Token Jaccard on raw tokens works for any language that tokenises on whitespace/alphanumeric boundaries — which covers most Indo-European languages. However, the stopword list in v0 is English-only. For a French golden set, 'le', 'la', 'les', 'est', 'sont' are not filtered, which means short prompts comparing only common French function words would score artificially high on Jaccard. For multilingual use, the STOPWORDS set must be extended per language. This is a documented

limitation.

Q9: A golden set passes all your checks. What are you still NOT guaranteeing?

A: Several important things. Coverage of the production query distribution: the golden set may contain structurally clean examples that don't reflect the diversity of real user queries. Factual correctness: no check verifies that expected answers are true. Metric appropriateness: PASS says nothing about whether exact match, ROUGE, or LLM judge is the right metric for this task. Pretraining contamination: the model may have memorised these examples. Annotation agreement: if multiple annotators produced the expected answers, inter-annotator agreement is not measured. PASS means no structural quality problems were detected — it does not mean the golden set will produce valid benchmark scores.

Q10: Why is INSUFFICIENT_INPUT separate from PASS rather than just being treated as PASS?

A: INSUFFICIENT_INPUT means a check could not run because required configuration was absent. Treating it as PASS would mislead: a report with INSUFFICIENT_INPUT on category_coverage and near_duplicate_inputs has not checked whether categories are balanced or whether prompts are near-duplicates. It has only confirmed that the checks that could run found nothing. INSUFFICIENT_INPUT in the overall status is a signal to the reviewer: 'these checks were skipped; consider providing the missing config before trusting this report as complete.'

4. Truth Boundary

GoldenSetAuditor does not:

• Score or evaluate model outputs — it audits the dataset, not the model

• Verify factual correctness of expected answers

• Detect pretraining contamination

• Measure inter-annotator agreement or annotation quality

• Check whether the golden set covers the production query distribution

• Handle multi-turn or context-dependent evaluation examples

GoldenSetAuditor does:

• Check seven structural and content quality failure modes before benchmark scores are published

• Return row-level findings with evidence — traceable to specific examples via id_col

• Produce a documented, reproducible audit attachable to evaluation documentation

• Distinguish structural violations (FAIL) from suspicious patterns (WARN)

5. Files Index

File	Purpose
src/goldensetauditor/core.py	7 checks, GoldenSetAuditConfig, GoldenSetFinding, GoldenSetReport
tests/test_goldensetauditor.py	13 unit tests across 4 test classes
scripts/generate_demo_reports.py	Demo with synthetic golden set quality issues
data/demo_golden_set.csv	Demo dataset — conflicting labels, weak answers, near-dups
README.md	Grouped arch diagram, About (circular validation problem), check table
CHANGELOG.md	v0.1.0 and v0.2.0 release notes
.github/workflows/ci.yml	Python 3.10/3.11/3.12 CI matrix
.github/workflows/publish.yml	OIDC PyPI trusted publisher on release

Resume-safe claim: Built GoldenSetAuditor, an evaluation dataset quality auditor for LLM/RAG applications that checks golden sets for conflicting expected answers, exact and near-duplicate prompts, weak reference answers, ambiguous questions, over-easy examples, and category coverage gaps, producing structured JSON/Markdown/HTML audit reports with per-finding

PASS/WARN/FAIL/INSUFFICIENT_INPUT status and explicit truth boundary.